Module 2: Web Development

Deploy websites using frameworks and platforms.

Estimated Time: 2 hours

NOTE: When counting from 1 to 5, we go in order: 1, 2, 3, 4, 5. Similarly, when following a step-by-step process, it's important to go through each step in sequence without skipping around. If you find yourself stuck at any point, don't hesitate to reach out to your instructor for help and support.

If a text is in orange, then it is a link. A blue text should be typed **as is**. Then all red texts should be replaced by the expected value. Just ensure that you typed in **without misspellings**.

Learning Task 4 – Creating a Frontend Web Application

Websites have been there for decades already but have evolved so much over the years. In the traditional monolithic architectures, the user interface (UI) is deeply integrated with both data and functionality—specifically all within one large PHP file. Typically, any changes in UI, data, or functionality requires modifying the specific file. As more features are added into the system, more pages (PHP files) are also added. Also, functional changes in one file might also require changes in another file. However, modern approaches started to leverage JavaScript to transfer interactivity from the server to the client-side (browser).

In mid 2000s, jQuery gained popularity among web developers and is still being used by legacy web systems worldwide even today. More recently, Node.js framework is now an integral part of most-used web tech stacks. JavaScript remains the bedrock in frontend web development and its statically typed superset, TypeScript, is a powerful enhancement in terms of long-term productivity and maintainability.

Node.js

Node.js was created to address the scalability limitations of traditional web servers by creating a runtime environment that could efficiently handle concurrent connections, particularly for real-time web applications. It is primarily used for its server-side capabilities and is crucial in the core development workflow of modern frontend web development. Now, let's install **Node.js**!

1. Launch your browser.
2. Go to Node.js (https://nodejs.org/en/download) website.
    - Select the version 22 or latest Long-term support version.
    - Scroll down a bit and ensure the correct platform and CPU architecture is selected.
    - Download the installer: ***msi*** (Windows) or ***pkg*** (macOS).
    - Once downloaded, complete the installation.

© Rodney M. Maniego Jr. v2.2026

3. Open the Command Prompt:
   - Check the version: `node -v`
   - Update NPM to latest: `npm install -g npm@latest`

Nuxt

As the limitations of traditional web development became more apparent, it enabled the rise of scalable web frameworks. One of which is Vue.js, which allows dynamic and reactive interfaces while promoting reusable code but avoiding rigid conventions. Vue.js is a web framework but requires a full-stack solution built on top of it. Nuxt emerged to simplify Vue.js development, offering a framework for streamlined server-side rendering and static site generation. It boosts developer efficiency and optimizes performance, notably for SEO and load times. Mainly, it is used for universal applications and is key to building scalable Vue.js projects. Let's get Nuxt running!

1. Check your "**secret project**" in the **LT2 Brand Ideation** submission.
   a. Use this project codename without spaces for simplicity, e.g. `bleedingheart`.
   b. Think about what this project is all about.
2. Launch **Command Prompt**:
   a. Navigate to your projects folder (or create a new one): `cd /d C:/projects/`
   b. Create a **Nuxt** project: `npm create nuxt@latest bleedingheart`
   c. If prompted, "**`Ok to proceed?`**", just hit **`ENTER`** key to continue.
   d. Wait for a while until it's completed.
   e. When prompted "**`Which package manager would you like to use?`**", select "`yarn`".
   f. When prompted "**`Are you interested in participating?`**", choose "`No`".
   g. When prompted " **`Initialize git repository?`**", choose "`Yes`".
   h. Open the new project repository: `cd bleedingheart`
3. Create a private GitHub repository of the same name, e.g. `bleedingheart`.
   a. Add "**rmaniego**" as collaborator.
   b. Copy the **SSH** clone URL.
4. Go back to the command prompt.
   a. Point existing project to remote repository, run the commands:

```
C:\projects\bleedingheart>git branch -M main

C:\projects\bleedingheart >git remote add origin URL

C:\projects\bleedingheart >git add -A

C:\projects\bleedingheart >git commit -m "[init] Initial project setup"

C:\projects\bleedingheart >git push -u origin main
```

5. Go to the LT4 Dashboard spreadsheet.

© Rodney M. Maniego Jr. v2.2026

   a. Input your website repository.
   b. Supply a brief description of your project.
6. Browse on the filled-out rows without a contributor.
   a. Select that project and set yourself as contributor (one contributor per repo only).
   b. Read the description of the project.
      i. Discuss with the owner if you need more details.
   c. Request the repository owner to invite you as collaborator.
   d. Accept the invitation.

Regulatory Compliance

As future software developers, we also need to account that software development is continually evolving. It now overlaps with other domains such as IT infrastructure and cybersecurity. Majority of the additional requirements is heavily tied in the software mishaps in the previous decades, including software oversight that threatened infrastructure and lives directly or indirectly. Stricter regulations and improved development practices aim to prevent these software failures.

Note: In this section, you may use VS Code or your favorite IDE (recommended). When using VS Code, install the "Vue - Official" extension to avoid configuration issues. Optionally, you may still use Notepad++ as your text editor.

1. Go back to your projects folder.
2. Clone your target repository where you are assigned as a contributor.
3. Open the cloned repository.
4. In the project root:
   a. Delete the "`app.vue`" file.
   b. Create the "`pages`" folder, this will define the routes in your web application.
   c. Create the "`components`" folder, this will contain reusable components.
5. Go inside the "`pages`" folder:
   a. Create the "`index.vue`" file.
   b. Create the "`terms.vue`" file.
   c. Create the "`policy.vue`" file.
   d. Create the "`support.vue`" file.
6. Similarly go inside the "`components`" folder:
   a. Create the "`Welcome.vue`" file.
   b. Create the "`Terms.vue`" file.
   c. Create the "`Policy.vue`" file.
   d. Create the "`Support.vue`" file.
   e. Create the "`NavBar.vue`" file.
   f. Create the "`Copyright.vue`" file.

© Rodney M. Maniego Jr. v2.2026

7. Go to **flaticons.com** or any similar services:
    a. Find a logo that fits the project description.
    b. Download and move PNG logo to the "`public`" folder.
8. Download a the reset.css file from meyerweb.com wesbite:
    a. Save the CSS file to the "`public`" folder.
9. Open with the "`pages/index.vue`" file:
    a. Copy the following contents to the file.

```
<script setup lang="ts">

import { useSeoMeta, useHead } from '@vueuse/head';

const title = "Code Name | Home";
const description = "Project description";

useSeoMeta({
  title: () => title,
  description: () => description,
  charset: "utf-8",
  viewport: "width=device-width, initial-scale=1.0"
});

useHead({
  link: [
    {rel: 'icon', type: 'image/png', href: '/logo.png'},
    {rel: 'stylesheet', href: '/reset.css'}
  ]
});</script>
```

    b. After the **script** tag, add the main **HTML** template.

```
<template>
  <client-only>
    <NavBar />
    <Welcome />
    <Copyright />
  </client-only>
</template>
```

    c. Notice the "`<Welcome />`" tag, it is referenced to the "`components/Welcome.vue`" file.
    d. After the **template** tag, add the scoped **style** tag and the style tag.

```
<style scoped></style>

<style></style>
```

    e. The style tag is used for elements dynamically created by JavaScript.

10. Repeat this to the remaining "`pages/*.vue`" files.
11. Push all of your changes.
    a. A good commit message could be: "`[pages] Add initial Vue application routes`"

© Rodney M. Maniego Jr. v2.2026

© Rodney M. Maniego Jr. v2.2026

Basic HTML

In order to build websites, core knowledge in HTML is essential. In traditional webpages, the "`<!DOCTYPE html>`" declaration is required for proper rendering of HTML5 in modern browsers. However, we now focus more on the main flexible components and dynamic structure of web pages when using web frameworks. It like creating small units and assembling it together based on the requirements.

1. Open the "`components/NavBar.vue`" file.
    a. Copy the following contents to the file.

```
<template>
  <client-only>
    <header>
      <nav>
          <a href="/">Home</a>
      </nav>
    </header>
  </client-only>
</template>

<style scoped></style>
```

    b. Push changes following our best practices.
2. Open the "`components/Copyright.vue`" file.
    a. Copy the following contents to the file.

```
<script setup lang="ts">
  const year = new Date().getFullYear();
</script>

<template>
  <client-only>
  <footer>
    <nav>
        <a href="terms">Terms and Conditions</a>
        <span> | </span>
        <a href="privacy">Privacy Policy</a>
        <span> | </span>
        <a href="support">Support</a>
    </nav>
    <p>
      <span>&copy; </span>
      <span>{{ year }}</span>
      <span> Company Name. All rights reserved.</span>
    </p>
  </footer>
  </client-only>
</template>

<style scoped></style>
```


© Rodney M. Maniego Jr. v2.2026

   b. Push changes following our best practices.
3. Open the "`components/Welcome.vue`" file.
   a. Copy the following contents to the file.

```
<script lang="ts">
if (typeof (window as any).global === "undefined") {
  (window as any).global = window;
}

export default {
  components: true,
};
</script>

<template>
  <client-only>
    <div id="textMessage" class="text-message">Hello, world!</div>
  </client-only>
</template>

<script setup lang="ts"></script>

<style scoped></style>

<style></style>
```

   a. Push changes following our best practices.
4. Repeat **Step 3** for the remaining "`components/*.vue`" files, except the **NavBar** and **Copyright**.
5. In the **Command Prompt** or integrated terminal:
   a. Install project dependencies: `corepack yarn install`
   b. Run the development server: `corepack yarn dev -o`
   c. Screenshot the main page of the website.
   d. Once done, submit it to your instructor via email.

To: **rodney.maniego.jr@vsu.edu.ph**
Subject: [2025-mm-dd] CSci 23 LT4 Surname

Sir Rodney,

Good day!

I am Juan D. Cruz (student #) of your CSci 23 class. Attached is the screenshot for your reference. Below is the link to my GitHub repository where I contributed to.

https://github.com/username/codename

Best regards,

Juan D. Cruz
BSCS-2

© Rodney M. Maniego Jr. v2.2026

6. Ensure proper submission for grading; confirm with your instructor if needed.
7. Delete all your files and empty the **Recycle Bin**. (Skip if your own PC)
8. Logout your GitHub, Gmail, Vercel, and VSUEE accounts. (Skip if your own PC)
9. Close the browser, Command Prompt, and other windows.
10. Shutdown the PC and make sure the AVR/UPS is turned off.
11. Arrange the chairs before leaving.

© Rodney M. Maniego Jr. v2.2026